

Graph-Based Analytics for Stock Market Dynamics: Integrating Neo4j, LangChain, and Financial Data

Maria Fernanda Cladera Melgar
MariaFernanda.CladeraMelgar@unil.ch

December 2024

1 Executive Summary

This project focuses on integrating graph databases and AI tools to analyze financial market data. By utilizing Neo4j for graph analytics and LangChain for natural language interaction, we built a system to examine stock relationships and sector dependencies. Working with the S&P 100 dataset, we applied methods like distance correlation and community detection to get insights into market dynamics. The work sets the stage for future expansion into larger datasets, real-time event analytics, and accessible interfaces, highlighting the potential of combining graph-based and AI-driven approaches in financial analysis.

2 Introduction

The analysis of financial markets often requires the integration of advanced analytical tools to uncover hidden relationships and provide actionable insights. This project presents a small-scale implementation that combines the capabilities of Neo4j, a graph database, with LangChain, an AI-driven natural language processing framework. By utilizing graph-based analytics, we aimed to explore the interconnectedness of financial market data, focusing on the S&P 100 dataset.

The primary goal of this project was to create a foundational framework that could scale into a more extensive application in the future. Initially, we modeled stock market relationships using graph-based methods, representing stocks as nodes and their correlations as edges. This graph structure allowed the use of advanced algorithms for centrality and community detection, revealing influential stocks and sector-level dynamics. To enhance user interaction, we integrated LangChain for natural language querying, enabling users to derive insights from the graph database in a user-friendly manner.

This project is the first step for a system that could be expanded to include global financial datasets, real-time event-driven analytics, and a user-friendly interface for broader applicability.

3 Literature Review

For our project, with the objective of creating a small-scale integration of financial market analysis, graph

databases, and AI-driven tools, we explore the intersection of financial data analysis, correlation techniques, and the application of graph-based methodologies.

3.1 Financial Market Analysis and Stock Behavior

Understanding stock behavior and market dynamics has been a cornerstone of financial research. Traditional approaches leverage statistical methods and models to analyze stock performance, evaluate sectoral indices, and study macroeconomic influences (Rojas et al., 2023). Financial markets exhibit complex interactions that require multidimensional analyses. Timothewt et al. (2021) emphasize the importance of integrating historical prices, sectoral performance, and external factors, such as environmental, social, and governance (ESG) metrics, into holistic models for better decision-making.

Incorporating AI and computational tools has significantly enhanced the depth and efficiency of financial data analysis. For instance, machine learning models and algorithms now facilitate pattern recognition in large datasets, paving the way for predictive analytics in portfolio optimization and risk assessment (Ada, Nfj, et al., 2020).

3.2 Correlation Analysis in Financial Systems

Central to our project's methodology is correlation analysis, which reveals relationships between financial variables, such as stock price movements, sector trends, or broader market indices. Pearson correlation remains a fundamental tool for linear relationships, while techniques like Granger causality offer insights into time-series data, providing temporal and predictive elements to financial analyses (Neo4j Technical Review, 2023).

In the context of this project, correlation analysis forms the basis for constructing meaningful connections in the dataset. For example, strong correlations between stocks in the same sector highlight intrinsic relationships that can be further explored using graph-based methodologies. The dynamic nature of financial data necessitates integrating real-time updates and more nuanced models, especially as markets react

to events such as earnings announcements or mergers (Rojas et al., 2023).

3.3 Graph Databases and Their Role in Market Analysis

Graph databases, such as Neo4j, are increasingly recognized for their ability to model complex financial systems. Unlike traditional relational databases, graph databases excel in representing and analyzing relationships, which is critical in financial systems characterized by interconnected entities like stocks, sectors, and indices (SurveyGraph Bases, 2022).

The integration of graph technology with market analysis allows for advanced queries that reveal hidden patterns and relationships. For example, the use of Louvain community detection in Neo4j helps cluster stocks into meaningful communities based on shared attributes or behaviors. This complements traditional statistical approaches by providing a structural and relational perspective on financial data (Timothewt et al., 2021).

3.4 Integrating Graph Q&A and LangChain in Financial Systems

To enhance the interactivity and utility of financial analysis, this project incorporates LangChain’s graph Q&A capabilities, enabling natural language querying of the graph database. This aligns with modern efforts to make complex financial analyses accessible to non-technical users (LangChain Graph Tutorial, 2025). LangChain provides a framework for integrating AI-driven question-answering systems with graph-based data, ensuring that users can seamlessly explore relationships, detect causality, and gain actionable insights from the dataset.

By combining the correlation analysis, graph databases, and LangChain technologies, this project demonstrates the potential for a small-scale system to offer dynamic, AI-powered insights into financial markets.

4 Data Collection

In this section, we present the processes for gathering, preprocessing, and preparing data for financial analysis.

4.1 Stock Market Data Extraction

4.1.1 S&P 100 Stock List

The first step in our data collection involves sourcing a comprehensive list of companies from the S&P 100 Index. This dataset serves as the foundation for our analysis, representing a significant portion of the market capitalization of the S&P 500. Using the Wikipedia library, we extract stock list information directly from the S&P 100 page.

To ensure data integrity and consistency:

- We adjusted naming conventions to handle special cases like `BRK-B` and `GOOGL`.
- We saved the cleaned list as `stocks.csv` for subsequent processing.

This dataset includes key identifiers like stock tickers and sector classifications, enabling downstream aggregation and analysis.

4.1.2 Financial Metrics from Yahoo Finance

To enrich the stock list with fundamental financial metrics, we use the Yahoo Finance API. Metrics we retrieve include:

- **Valuation Metrics:** Market capitalization.
- **Profitability Metrics:** Profit margins.
- **Risk Metrics:** Beta values and 52-week change.

To standardize the data, we replace missing values with zeros and apply z-score normalization to ensure uniformity across metrics.

4.1.3 Historical Price Data

Also using the YFinance API, we collected historical price data for all S&P 100 companies over a five-year period. The dataset includes information on the Open, High, Low, and Close prices for each stock. The Close price was used to compute the logarithmic daily returns (`DailyLogReturn`) for each stock using the formula:

$$r_t = \ln \left(\frac{P_t}{P_{t-1}} \right)$$

The resulting dataset contains the stock ticker (`Symbol`), the date, and the corresponding daily logarithmic return (`DailyLogReturn`) for each stock.

4.2 Exploratory Data Analysis

We examined various aspects of the S&P 100 data to get preliminary insights into its structure and behavior.

Distribution of Returns: The Figure 1 illustrates the distribution of daily logarithmic returns across all stocks in the dataset. The data follows a normal distribution, centered around a mean of 0.02, with a standard deviation of 1.00. The majority of the returns are concentrated near zero, reflecting the typical behavior of stock returns, while extreme positive and negative returns are less frequent.

Cumulative Log Returns by Sector: We want to evaluate the trends of each sector over time. Figure 2 presents that sectors most of the sectors have a positive trend; sectors like Information Technology and Consumer Discretionary are exhibiting notable growth. While the Real State presents a flat behavior.

Number of Companies per Sector: After a review of the trend over time, we want to understand

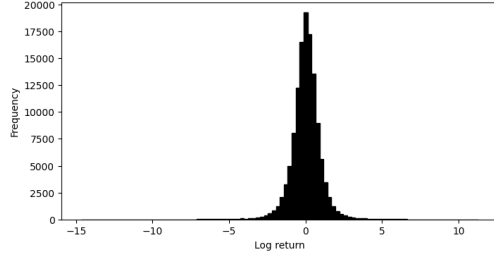


Figure 1: Distribution of Daily Log Return

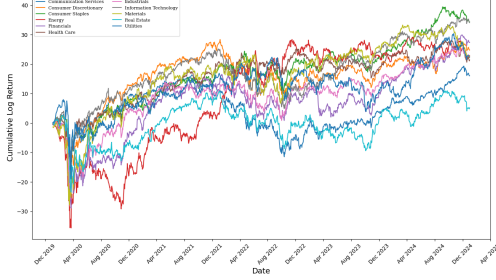


Figure 2: Cumulative Log Returns by Sector

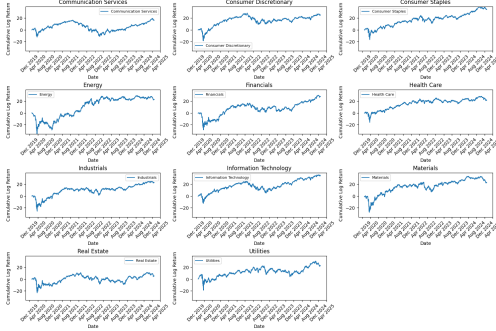


Figure 3: Cumulative Log Returns each Sector

how many companies are in each sector. In figure 4 we can see a dominance of Information Technology (18/100), Financials (15/100), and Health Care (14/100) in terms of company count.

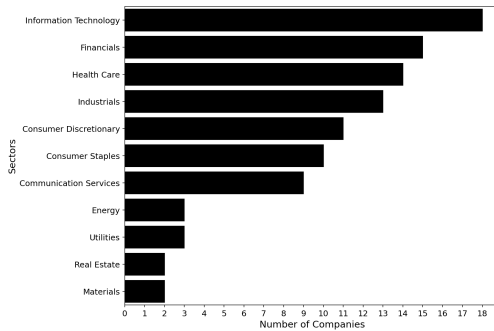


Figure 4: Number of Companies by Sector

Market Capitalization: We categorize stocks into blue chip, large cap, mid cap, and small cap classes based on most recent values of market capitalization (the total value of a company’s shares in the stock market). Blue chips, with market caps exceeding \$1 trillion, represent global industry leaders known for sta-

bility and reliability, here we have companies like Apple, Amazon, Broadcom, Alphabet, Meta, Microsoft, Nvidia and Tesla. Large caps (\$100 billion–\$1 trillion) dominate the dataset, reflecting the S&P 100’s focus on established market players, some examples of this companies are AbbVie, Abbot Laboratories, Accenture, Abde and AMD. Mid caps (\$50 billion–\$100 billion) and small caps (<\$50 billion) are less represented, emphasizing the index’s preference for financially resilient and globally influential companies.

Position	Number of Companies
Blue Chip	8
Large Cap	67
Mid Cap	20
Small Cap	5

Table 1: Number of Companies by Market Cap

After this preliminary review of our dataset, we continue with the calculations of the Correlation for the main analysis.

4.3 Correlation Analysis

We employed distance correlation, a metric designed to capture both linear and non-linear relationships. Unlike the traditional Pearson correlation, which only accounts for linear dependencies, distance correlation provides deeper insights into complex, non-linear interactions between stocks. This approach was inspired by the paper Distance Correlation Market Graph: The Case of S&P500 Stocks by Ugwu et al. (2023), which highlights the advantages of distance correlation in representing financial relationships. The steps we followed were:

- **Calculate the Distance Correlation Matrix:** To analyze dependencies across stocks, we computed a symmetric matrix where each entry represents the aggregated distance correlation between two stocks over the entire time period. Aggregating over the entire dataset provides a more stable and robust understanding of long-term correlations, minimizing the influence of short-term noise.
- **Data Transformation:** We transformed the symmetric matrix into a long-format table. In this format, each row corresponds to a unique stock pair and their computed distance correlation value.

Finally we saved the correlation matrix for the graph data base construction.

5 Stock Market Graph Construction

To enhance the analysis of S&P 100 stocks, we utilized graph-based methods to model relationships between

stocks. By representing stocks as nodes and their dependencies as edges, we enabled the application of advanced algorithms for centrality and community detection.

5.1 Data Preparation

We imported financial data, including stock metadata and pairwise distance correlation values. Correlations exceeding a threshold of 0.6 were retained to focus on meaningful relationships. This threshold was chosen to balance granularity and clarity in graph representation.

5.2 Node and Relationship Creation

Using Neo4j, we constructed a graph database where each stock was represented as a node with attributes such as `symbol`, `name`, and `sector`. We also modeled sectors as distinct nodes and linked stocks to their respective sectors using `BELONGS_TO` relationships. Correlations between stocks were encoded as `CORRELATED` relationships with a `correlation` property.

In figure 5, we present the main Stock Market Graph along with the two primary sectors, Information Technology and Financial, including all nodes and their relationships.

5.3 Graph Projection

We utilized the `gds.graph.project` function from Neo4j’s Graph Data Science (GDS) library to project our graph into the GDS workspace under the name `stockGraph`. This approach was chosen due to its efficiency in preparing the graph for subsequent analysis, such as centrality measures and community detection. The projection process ensures that only the necessary nodes and relationships are included optimizing computational performance for large-scale graph operations (in the case of scalability of the analysis).

The `stockGraph` projection consists of 100 nodes labeled as Stock and 318 relationships. To continue with the analysis, we decided to focus on two main algorithms: Centrality Analysis and Community Detection.

6 Centrality and Community Detection Analysis

Building on the previous steps, we focused on two key analyses: Centrality Analysis and Community Detection. These methods helped identify influential stocks and uncover clusters with similar behaviors. Centrality metrics to highlighted the most important stocks, while community detection revealed groups of stocks with stronger internal relationships, providing insights into sector dynamics.

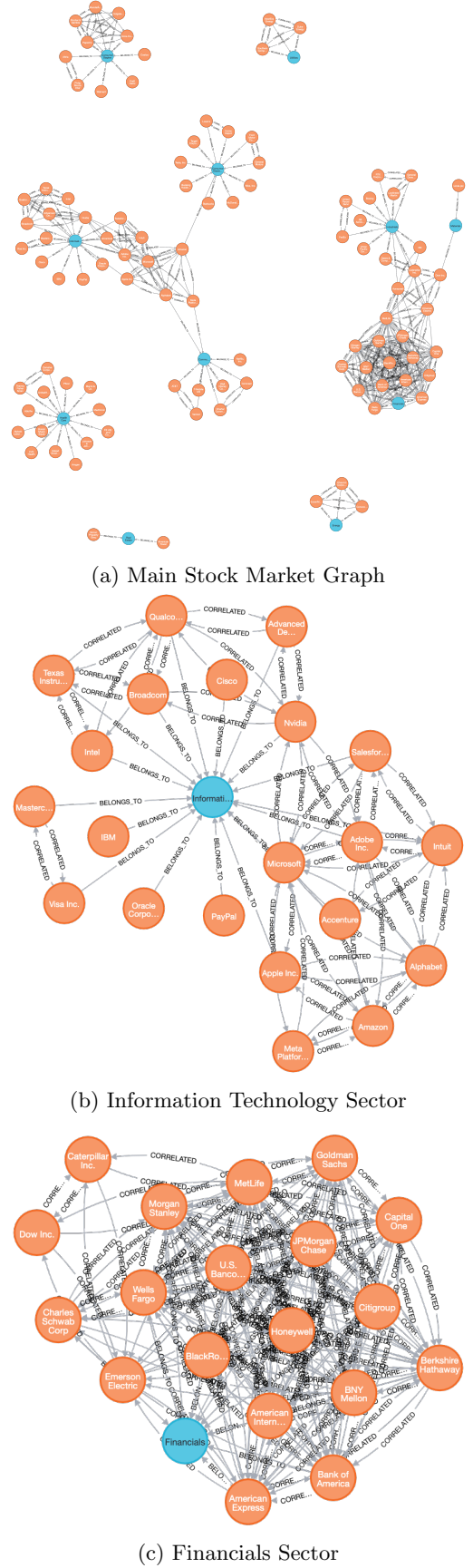


Figure 5: Stock Market Graph Database

6.1 Centrality Analysis

We applied three centrality metrics—Eigenvector Centrality, Betweenness Centrality, and Degree Centrality—to identify key stocks within the market network. These metrics offer insights into each stock’s influence based on its connections and position in the graph.

6.1.1 Eigenvector Centrality

We applied Eigenvector Centrality to our graph using the `gds.eigenvector.mutate` function from the Neo4j Graph Data Science (GDS) library. Eigenvector centrality is a measure of the influence of a node in a network, where the influence is not just based on the number of direct connections, but also on the influence of the nodes it is connected to. In financial markets, stocks with high eigenvector centrality are typically those that are well-connected to other influential stocks, making them central to market behavior and dynamics (Bonacich, 2007).

To ensure the algorithm’s accuracy, we set a maximum iteration limit of 100, and the algorithm converged after 18 iterations, indicating it reached a stable solution within the provided limit. The results show that stocks with the highest eigenvector centrality values predominantly belong to the financial sector, including JPMorgan Chase (JPM), MetLife (MET), and BNY Mellon (BK). These results suggest that the financial sector contains stocks with strong connections within the market, possibly indicating their influence in driving broader market trends (See Figure 6 and Table 2).

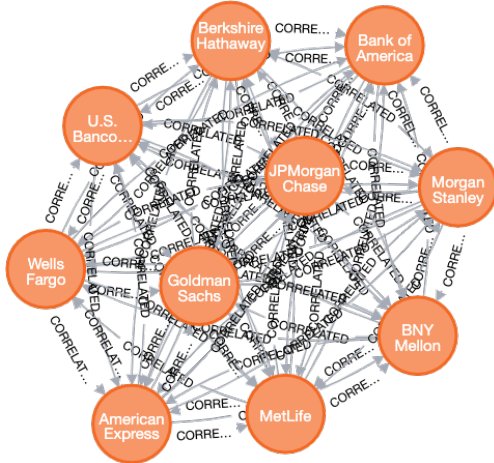


Figure 6: Top 10 Stocks with Higher Eigenvector Centrality

The results show that JPMorgan Chase has the highest eigenvector centrality value of 0.28, closely followed by MetLife at 0.28. This indicates that these two financial companies are particularly well-connected to other important players in the market. Other major financial institutions such as BNY Mellon, Morgan Stanley, and Bank of America also appear in the top ranks, demonstrating the significance of the financial sector in the S&P 100 network.

Name	Value
JPMorgan Chase	0.283733
MetLife	0.282753
BNY Mellon	0.276688
Morgan Stanley	0.276688
Bank of America	0.276688
Berkshire Hathaway (CB)	0.276411
U.S. Bancorp	0.267151
Wells Fargo	0.267151
Goldman Sachs	0.265567
American Express	0.263075

Table 2: Top 10 Stocks by Eigenvector Centrality in the Financial Sector

Similarly, the eigenvector centrality values for the top 10 companies show only minor differences in scores, with most companies having values between 0.26 and 0.28. This suggests that, although these companies are influential, their relative influence within the network is relatively similar.

We also wanted to see which other sectors were influential in the network. The financial sector stands out with the highest average eigenvector centrality, indicating its dominant role in the S&P 100. In contrast, sectors like Materials, Industrials, and Information Technology show lower centrality values, while Health Care and Real Estate have the lowest, reflecting less connectivity (See Table 3).

Sector	Value
Financials	0.2524
Materials	0.0162
Industrials	0.0144
Information Technology	1.0381e-8
Communication Services	6.1196e-9
Consumer Discretionary	2.7137e-9
Consumer Staples	1.0780e-11
Energy	4.0407e-15
Utilities	4.0407e-15
Health Care	5.3163e-21
Real Estate	1.9055e-170

Table 3: Average Eigenvector Centrality by Sector

6.1.2 Betweenness Centrality

As second centrality metric, We applied Betweenness Centrality using the `gds.betweenness.mutate` function. Betweenness centrality measures the extent to which a node lies on the shortest paths between other nodes in the network. In the context of financial markets, stocks with high betweenness centrality are crucial for information flow within the network, as they act as intermediaries between other stocks.

The algorithm’s results show that MetLife (MET) has the highest betweenness centrality value of 111.38, followed by major technology stocks such as Nvidia (NVDA) and Microsoft (MSFT). These companies have high betweenness centrality, indicating that they

play a significant role in bridging different sectors and facilitating market information transfer. This suggests that these stocks not only influence their direct connections but also serve as key intermediaries in connecting diverse parts of the market (See Figure 7 and Table 4).

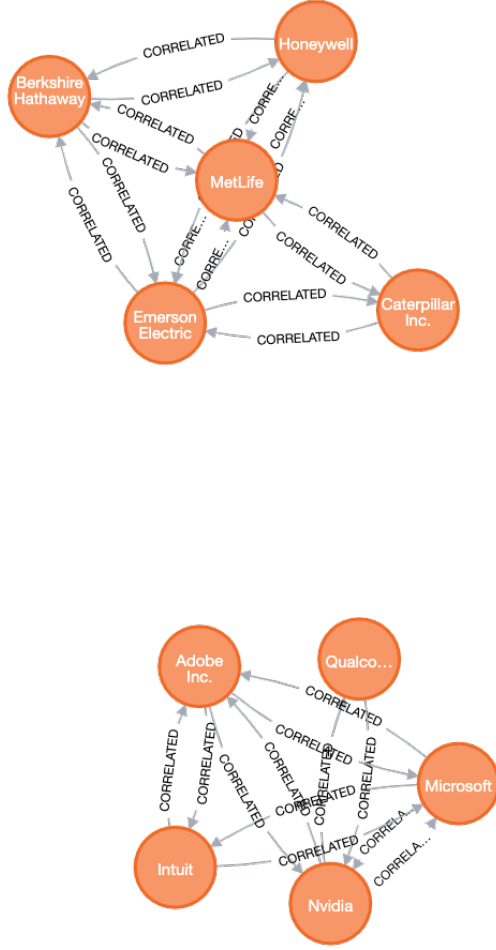


Figure 7: Top 10 Stocks with Higher Betweenness Centrality

Symbol	Sector	Value
MET	Financials	111.381
NVDA	Information Technology	91.000
MSFT	Information Technology	67.000
CAT	Industrials	38.000
HON	Industrials	38.000
QCOM	Information Technology	36.000
BRK-B	Financials	34.000
ADBE	Information Technology	33.000
INTU	Information Technology	26.667
EMR	Industrials	19.167

Table 4: Top 10 Stocks by Betweenness Centrality

6.1.3 Degree Centrality

To analyze the Degree Centrality metric, we'll follow a similar approach as we did with the Eigenvector and Betweenness Centrality metrics. Degree centrality is a measure of how many direct connections a node has in

a network. In the context of stocks, this means that stocks with a higher degree centrality are connected to many other stocks, making them potentially more influential within the network.

The results show that MetLife (MET) has the highest degree centrality value of 17, followed by JPMorgan Chase (JPM) and Berkshire Hathaway (BRK-B), which both have a degree centrality score of 15. This indicates that these financial stocks are well-connected to other companies in the network, suggesting their pivotal role in the market's functioning. The concentration of high degree centrality stocks in the Financials sector further emphasizes the sector's key position in the market's interconnections (See Figure 8 and Table 5).

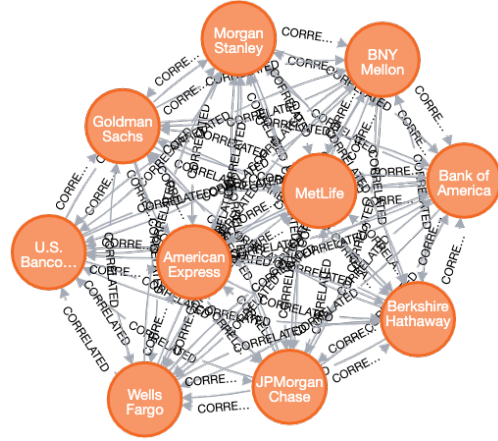


Figure 8: Top 10 Stocks with Higher Degree Centrality

Name	Value
MetLife	17.0
JPMorgan Chase	15.0
Berkshire Hathaway (Class B)	15.0
Bank of America	14.0
BNY Mellon	14.0
Morgan Stanley	14.0
Goldman Sachs	13.0
U.S. Bancorp	13.0
American Express	13.0
Wells Fargo	13.0

Table 5: Top 10 Stocks by Degree Centrality in the Financial Sector

6.2 Community Detection

In this section, we apply community detection techniques to uncover groups of stocks that exhibit strong internal relationships. We divided the community detection analysis into two key steps: component identification followed by Louvain community detection. This helps in understanding the clustering behavior of stocks, critical for detecting market segments that behave similarly or are interconnected through various financial relationships. The goal is to identify key

clusters and gain insights into the structure of the S&P 100 stock network.

6.2.1 Component Identification (Weakly Connected Components)

We began by using the weakly connected components (WCC) algorithm to identify distinct connected components within the graph. This step was essential for the following reasons:

- **Excluding isolated nodes:** The WCC algorithm helped isolate the most relevant subgraphs by excluding isolated nodes. By focusing only on connected stocks, we ensured that the community detection process would be based on meaningful relationships.
- **Improved computational efficiency:** Identifying the largest connected component before applying community detection helped reduce the size of the graph, allowing the Louvain algorithm to be applied efficiently. This ensured that the analysis was performed on a manageable and impactful subset of the data.

As a result, we identified 48 weakly connected components in total, and chose to focus on the largest component, which contained 21 stocks. This component represented the most connected group of stocks in the dataset, providing the most valuable insights (See Table 6).

C.ID	C.Size	Stocks
5	21	AIG, AXP, BAC, BK, BLK...
0	15	AAPL, ACN, ADBE, AMD...
22	5	CL, KO, MDLZ, PEP, PG
25	3	COP, CVX, XOM
35	3	DUK, NEE, SO
39	3	GD, LMT, RTX
21	2	CHTR, CMCSA
32	2	DHR, TMO
37	2	F, GM
38	2	FDX, UPS
45	2	HD, LOW
58	2	MA, V
65	2	MO, PM
85	2	T, VZ
1	1	ABBV
2	1	ABT

Table 6: Weakly Connected Components with Stocks

6.2.2 Louvain Community Detection

We applied the Louvain algorithm to detect communities in the network by maximizing modularity. This method identifies groups of stocks that are more strongly interconnected with each other than with the rest of the network. The results revealed two main communities in the S&P 100 stock network.

The first community (Community 12), shown in Figure 9, is made up of financial sector companies such as JPMorgan Chase, Bank of America, and Wells Fargo. The dense connections between these stocks suggest that the financial sector is highly interconnected, likely sharing common market dynamics that influence their collective behavior.

The second community (Community 9), shown in Figure 10, includes a mix of stocks from various sectors, such as Honeywell, Caterpillar Inc., and Emerson Electric. This community is less tightly interconnected compared to the first one, which indicates that stocks in this group might be influenced by a broader range of factors across different market segments.

The clear separation between financial stocks and other sectors emphasizes the financial sector's significant role in the market dynamics.

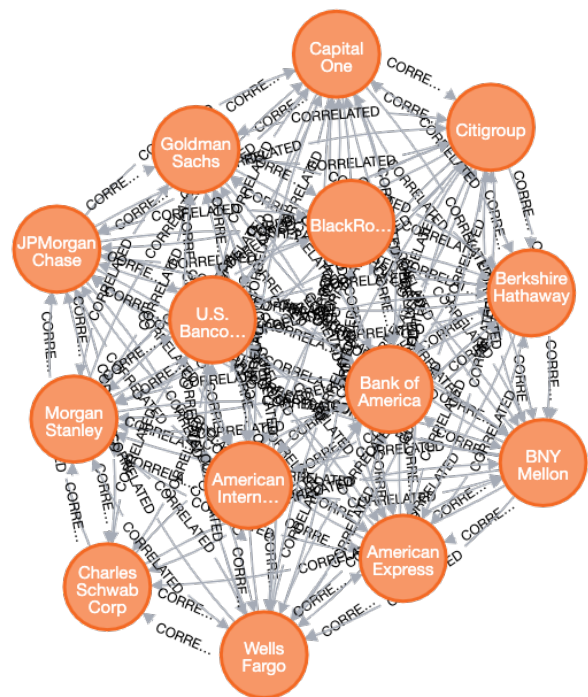


Figure 9: Community 12: Financial stocks

We used graph algorithms to analyze S&P 100 stocks, focusing on centrality metrics and community detection. The results revealed key stocks and sectors, with the financial sector being particularly influential. Next, we will use these insights to build a Question Answering system integrated with the graph database to provide answers based on stock relationships and network dynamics.

7 Integration with LangChain for Question Answering

In this section, we will integrate LangChain with a graph-based approach to query and analyze data. We followed the LangChain graph tutorial as a reference for the implementation. The code is structured into two main approaches: one simple, focusing on basic

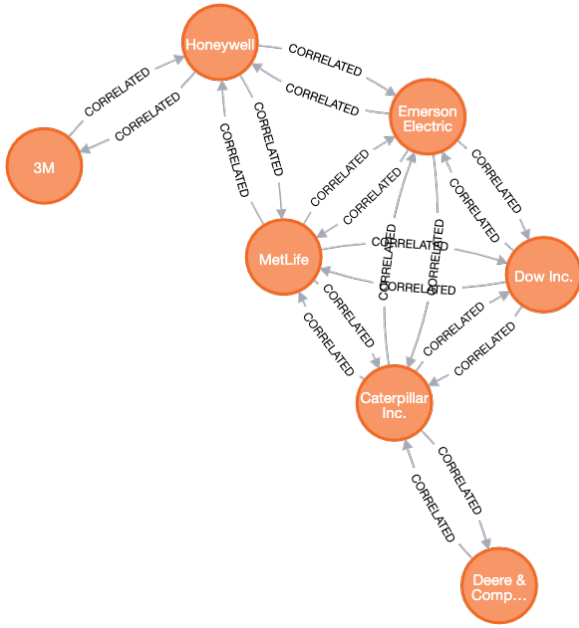


Figure 10: Community 9: Stocks from various sectors

functionalities, and another more complex, with additional controls to enhance flexibility and control over the querying process. The first approach sets up a basic graph-based querying system, while the second introduces various parameters and conditions to fine-tune the model's performance, enabling the extraction of meaningful insights from the knowledge graph.

7.1 Basic Cypher Query Execution with LLM Integration

First, we focus on the integration of the Neo4j graph database with LangChain's language model-powered query system, aiming to streamline querying of stock data stored within the graph.

The process begins with establishing a connection to the Neo4j database and the OpenAI API (using the GPT-4 model). The GraphCypherQChain is then initialized to automatically generate Cypher queries based on user input. This setup allows users to ask natural language questions, such as "What are the top 10 stocks by eigenvector centrality?", and receive results in a user-friendly format (See Figure 12).



Figure 11: Diagram of LLM integration

7.2 Advanced Implementation with LangGraph

In this second section, we develop a more advanced implementation of LangChain's graph-based query system, building upon the previous setup to enhance the

```
responsechain.invoke({"query": "What are the top 10 stocks by eigenvector centrality?"})
response
✓ 20s

> Entering new GraphCypherQChain chain...
Generated Cypher:
name
MATCH (s:Stock)
RETURN s.name, s.name, s.eigenvectorCentrality
ORDER BY s.eigenvectorCentrality DESC
LIMIT 10

Full Context:
[{"s.symbol": "JPM", "s.name": "JPMorgan Chase", "s.eigenvectorCentrality": 8.2627289282377863}, {"s.symbol": "MET", "s.name": "MetLife", "s.eigenvectorCentrality": 8.2627289282377863}]

> Finished chain.

{"query": "What are the top 10 stocks by eigenvector centrality?",
 "result": "The top 10 stocks by eigenvector centrality are JPMorgan Chase, MetLife, Bank of America, Berkshire Hathaway (Class B), U.S. Bancorp, Wells Fargo, ..."}
Python
```

Figure 12: Query generation and output

capabilities and accuracy of the system, this also following the LangChain tutorial. This approach incorporates additional components and introduces a more complex decision-making process, ensuring that the generated Cypher queries are validated, corrected, and executed in an intelligent and structured manner.

7.2.1 Main Components

TypedDicts for Input and Output States: We define InputState, OverallState, and OutputState using TypedDict, which are essential data structures that facilitate the flow of information through the LangChain pipeline. The InputState holds the user's query, while the OverallState manages the evolving context and tracking of steps in the process. OutputState contains the final answer and the steps executed.

Guardrails for Question Classification: The first step in the process involves classifying the user's question to determine whether it is related to stocks or finance. The guardrails system acts as a decision-making filter, utilizing a custom model to decide if the query should proceed or be discarded. If the question is deemed unrelated to stocks, the process terminates; otherwise, it continues with generating a Cypher query.

Example Selection and Cypher Query Generation: To generate Cypher queries from natural language, we use semantic similarity to select the most relevant examples from the question database. The example selector leverages OpenAI embeddings and a FAISS vector store to find the closest examples. The system then translates the user's question into a syntactically correct Cypher query using the text2cypher chain, which is informed by the schema and prior examples.

Cypher Validation: Once a Cypher query is generated, the system performs validation to ensure its correctness. The query is checked for syntax issues, undefined variables, missing node labels, and relationships. If any errors are found, the query is sent to the correct cypher step, where it is revised to ensure it aligns with the schema and query requirements.

Execution of the Cypher Query: After validation and correction, the Cypher query is executed against the Neo4j graph database. The results are stored in the database records field. If no relevant data is returned, a fallback message is provided

stating that no relevant information was found in the database.

Final Answer Generation: After executing the query, the results are passed to the generate final answer step, where they are used to form a human-readable response. This answer is generated by the language model to ensure the output is accurate and user-friendly.

The system is organized using a StateGraph model, where each step of the process is represented as a “node” (a stage in the workflow). These steps include things like checking the question (guardrails), turning it into a Cypher query (generate Cypher), checking the query for errors (validate Cypher), and running the query on the database (execute Cypher). The system moves from one step to the next depending on the situation—for example, it might fix any mistakes in the query before running it or provide an answer once the query has been executed. This setup helps ensure that the process flows smoothly, from the moment you ask a question to receiving the final answer.

The flow is visually represented through a diagram, which illustrates the sequence of steps and how they are interconnected (See Figure ??).

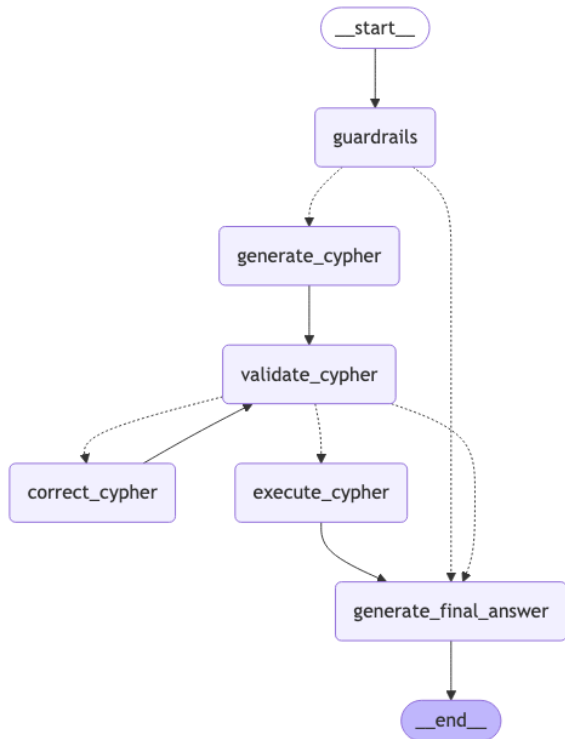


Figure 13: LangChain Process Flow

8 Results and Discussion

This project aimed to explore the integration of graph databases and AI-powered question-answering systems, with a focus on financial market analysis. By leveraging Neo4j for graph analytics and LangChain

for natural language interaction, we created a mini-scale system that demonstrates the potential of these tools in uncovering relationships, extracting insights, and answering domain-specific queries. This discussion evaluates the relevance, challenges, and potential applications of the system.

8.1 Relevance of the Project

Graphs provide an intuitive and powerful framework for analyzing interconnected data. In the financial market, understanding the relationships between stocks, sectors, and events is crucial for portfolio management and risk assessment. Our integration of Neo4j allowed us to visualize and analyze stock relationships based on centrality, correlation, and community detection. Coupled with LangChain’s natural language capabilities, the system enables users to extract actionable insights without requiring technical expertise in query languages.

8.2 Key Insights

The centrality analysis revealed that companies such as JPMorgan Chase and MetLife play pivotal roles in their respective networks, emphasizing the dominance of certain stocks in influencing market trends. Community detection highlighted distinct groupings, particularly within the financial and technology sectors, which reflect real-world sector interdependencies. Additionally, the successful integration of LangChain demonstrated the feasibility of AI-driven querying within graph databases, making complex insights accessible through simple questions.

8.3 Future Directions

To enhance the utility and scalability of this project, we will continue working on:

1. **Expanded Dataset:** Incorporating a broader dataset, such as the S&P 500 or global indices, would provide a more comprehensive market view.
2. **UI Development:** Building a user-friendly interface would enable non-technical users to interact seamlessly with the system.
3. **Causality Analytics:** Integrating real-time financial events to identify causality and predict market shifts.

9 Conclusion

In this project, we explored the integration of graph databases and AI tools to analyze financial markets on a smaller scale. By leveraging Neo4j’s graph-based analytics, we examined relationships between stocks, using metrics like centrality and community detection to uncover sectoral connections and identify influential stocks. With the integration of LangChain, we added

```

langgraph.invoke({"question": "How many stocks belong to the Information Technology sector?"})
✓ 3.5s Python

Invoking guardrails_chain with question: How many stocks belong to the Information Technology sector?
Guardrails output: decision='stock'

{'answer': 'There are 18 stocks that belong to the Information Technology sector.',
'steps': ['guardrail',
'generate_cypher',
'validate_cypher',
'execute_cypher',
'generate_final_answer'],
'cypher_statement': "MATCH (s:Stock)-[:BELONGS_TO]->(:Sector {name: 'Information Technology'}) RETURN count(DISTINCT s)"}

```

Figure 14: Query Example 1

```

langgraph.invoke({"question": "Which stocks have a correlation with AAPL?"})
✓ 3.5s Python

Invoking guardrails_chain with question: Which stocks have a correlation with AAPL?
Guardrails output: decision='stock'

{'answer': 'The stocks that have a correlation with AAPL are AMZN, GOOGL, and MSFT.',
'steps': ['guardrail',
'generate_cypher',
'validate_cypher',
'execute_cypher',
'generate_final_answer'],
'cypher_statement': "MATCH (a:Stock {symbol: 'AAPL'})-[:CORRELATED]->(b:Stock) RETURN b.symbol"}

```

Figure 15: Query Example 2

an interactive layer, enabling intuitive question-and-answer functionality that made insights accessible even to non-technical users.

We chose the S&P 100 dataset as a manageable yet insightful starting point, allowing us to focus on understanding the interconnectedness of stocks and the relationships within sectors. By applying distance correlation, we captured both linear and non-linear dependencies, complementing the structural insights provided by graph analytics.

In this work, we focus on the scalability and potential of the system. Expanding the dataset to include broader indices, incorporating event-driven data, and developing a user-friendly interface are clear next steps to enhance the tool’s functionality. We also see opportunities to integrate real-time causality analysis, which could enable predictive capabilities and provide actionable intelligence for market participants.

10 References

- LangChain Team. (n.d.). *LangChain Graph Tutorial*. Retrieved from <https://python.langchain.com/docs/tutorials/graph/>
- TimotheWT. (n.d.). SP100 Analysis with GNNs: Data Collection and Preprocessing. Retrieved from https://github.com/timothewt/SP100AnalysisWithGNNs/blob/master/notebooks/1-data_collection_and_preprocessing.ipynb
- Ada, N. F. J., et al. (2020). Mathematics-based approaches for analyzing financial data using graph databases. *Mathematics*, 11(3832).
- Technical Review Team. (n.d.). *Graph-Based Approaches for Stock Market Insights*. [Unpublished Document].

- SurveyGraphBases Authors. (n.d.). *Graph Databases: Theory and Applications in Financial Markets*. Retrieved from [Unpublished Document].
- Neo4j. (n.d.). *Graph Data Science Documentation*. Retrieved from <https://neo4j.com/docs/graph-data-science/current/>
- EcoFinanceN Authors. (n.d.). *EcoFinance in Market Analysis*. [Unpublished Document].
- General Financial Analytics Team. (n.d.). *Correlations in Financial Data: Techniques and Insights*. Retrieved from [Unpublished Document].